

C++ Programming Questions and Solutions

Complete collection of multiple choice questions with detailed explanations

Question 1: Prime Counting Methods

Which of the following methods are used to count the prime numbers between 1 and $n-1$ (both inclusive)?

- A) Naïve Approach
- B) Sqrt Approach
- C) Sieve of Eratosthenes
- D) All of these

Answer: D) All of these

Explanation:

- **Naïve Approach:** Check each number from 2 to $n-1$, and for each number, check if it's divisible by any number from 2 to that number minus 1. Time complexity: $O(n^2)$
 - **Sqrt Approach:** Improvement over naïve - only check divisibility up to square root. Time complexity: $O(n\sqrt{n})$
 - **Sieve of Eratosthenes:** Use boolean array to mark multiples of each prime. Time complexity: $O(n \log \log n)$
-

Question 2: Why 1 is Not Prime

Why 1 is not considered as the prime number?

- A) It has only one factor
- B) It has two factors i and $-i$
- C) It is a prime number
- D) None of these

Answer: A) It has only one factor

Explanation: A prime number is defined as a number greater than 1 that has exactly two distinct positive factors: 1 and itself. The number 1 has only one positive factor (1), so it doesn't meet the definition of a prime number.

Question 3: Naïve Approach Time Complexity

In the naïve approach for calculating total number of primes between 1 and $n-1$ (both inclusive), what is the time complexity for deciding whether the i -th number is prime or not?

- A) $O(n)$
- B) $O(n^2)$
- C) $O(n \log(\log n))$
- D) $O(\sqrt{n})$

Answer: D) $O(\sqrt{n})$

Explanation: In the naïve approach, to check whether the i -th number is prime, we test whether i is divisible by any number from 2 to $\sqrt{i}$. This is because if i has a factor larger than $\sqrt{i}$, it must also have a smaller corresponding factor. So checking up to $\sqrt{i}$ is sufficient.

Question 4: Sqrt Approach Total Time Complexity

What is the time complexity to calculate total number of primes between 1 and $n-1$ (both inclusive), using sqrt approach?

- A) $O(n)$
- B) $O(n^2)$
- C) $O(n \log(\log n))$
- D) $O(n\sqrt{n})$

Answer: D) $O(n\sqrt{n})$

Explanation: Using the sqrt approach:

- Loop through each number from 2 to $n-1 \rightarrow O(n)$ iterations
 - For each number i , check if it is prime by testing divisibility up to $\sqrt{i} \rightarrow O(\sqrt{n})$ per number
 - Total time complexity: $O(n \times \sqrt{n}) = O(n\sqrt{n})$
-

Question 5: Correct isPrime Implementation

Which of the following is the correct implementation for `isPrime()` function in naïve approach?

- A) `bool isPrime(int n){ if(n <= 1) return false; for(int i=2; i<n; i++) if(n%i) return false; return true; }`

- B) `bool isPrime(int n){ if(n <= 1) return -1; for(int i=2; i<n; i++) if(n%i==0) return false; return true; }`
- C) `bool isPrime(int n){ if(n <= 1) return false; for(int i=2; i<n; i++) if(n%i==0) return false; return true; }`
- D) `bool isPrime(int n){ if(n <= 1) return true; for(int i=2; i<n; i++) if(n%i == 0) return true; return false; }`

Answer: C)

Explanation:

```
cpp

bool isPrime(int n){
    if(n <= 1) return false;    // 0 and 1 are not prime numbers
    for(int i = 2; i < n; i++){
        if(n % i == 0)          // If divisible by any i, it's not prime
            return false;
    }
    return true;                // No divisors found, it's prime
}
```

Question 6: Sieve of Eratosthenes Time Complexity

What is the time complexity of sieve of eratosthenes?

- A) $O(n)$
- B) $O(n^2)$
- C) $O(n \log(\log n))$
- D) $O(\sqrt{n})$

Answer: C) $O(n \log(\log n))$

Explanation: The Sieve of Eratosthenes works by marking multiples of each prime. Each prime p marks about n/p numbers as not prime. The total operations over all primes is $n(1/2 + 1/3 + 1/5 + \dots) = O(n \log(\log n))$.

Question 7: Optimized Sieve Time Complexity

What is the time complexity of optimised sieve of eratosthenes?

- A) $O(n)$
- B) $O(n^2)$

- C) $O(n \log(\log n))$
- D) $O(\sqrt{n})$

Answer: C) $O(n \log(\log n))$

Explanation: The optimized sieve improves by starting from p^2 instead of $2p$, but the asymptotic complexity remains $O(n \log \log n)$. The optimization only reduces constant factors.

Question 8: Segmented Sieve First Multiple

In segmented sieve algorithm, how do we calculate the first multiple of a prime p between numbers l and h ?

- A) `first_multiple = (l/p)*p;`
- B) `first_multiple = (1/p)*p; if(first_multiple < l) first_multiple += p;`
- C) `first_multiple = (1/p)*p; if(first_multiple < l) first_multiple += p;`
- D) None of these

Answer: D) None of these

Explanation: The correct formula is: `first_multiple = max(p * p, ((l + p - 1) / p) * p);`

- If $p^2 \geq l$, start from p^2
 - Otherwise, find the first multiple of $p \geq l$ using ceiling division
-

Question 9: Segmented Sieve Code Error

Below is the part of segmented sieve algorithm code. Can you spot which line has an error or some scope of improvement?

```
1. for(auto p: basePrimes){
2. int fm = (l/p)*p;
3. if(fm < l) fm += p;
4. int = fm;
5. while(<=rK
6. segSieve[j] = false;
7. +p;
8. )
9. )
```

- A) 4 only
- B) 6 only

- C) 4 and 6 both
- D) None

Answer: A) 4 only

Explanation: Line 4: `int = fm;` is invalid syntax. Should be `int j = fm;` Line 6 is correct - it marks the index as non-prime.

Question 10: Sieve Optimization

Which of the following is done for Optimisation in sieve of eratosthenes?

- A) inner loop starts from $j = i^2$; outer loop end at $i == n$;
- B) inner loop starts from $j = i^2$; outer loop end at $i == \sqrt{n}$;
- C) inner loop starts from $j = i$; outer loop end at $i == \sqrt{n}$;
- D) inner loop starts from $j = i$; outer loop end at $i == n$;

Answer: C) inner loop starts from $j = i$; outer loop end at $i == \sqrt{n}$;

Explanation:

- Inner loop from i : All smaller multiples would already be marked by smaller primes
 - Outer loop to $\sqrt{n}$: Beyond $\sqrt{n}$, all non-primes would have already been marked
-

Question 11: Exponentiation Time Complexity

What is the worst case time complexity to calculate a^b in optimised manner?

- A) $O(b)$
- B) $O(a)$
- C) $O(\log b)$
- D) $O(\log a)$

Answer: C) $O(\log b)$

Explanation: Using exponentiation by squaring (binary exponentiation), we halve the exponent b at each step, resulting in $O(\log b)$ time complexity instead of naive $O(b)$.

Question 12: Exponentiation Pseudocode

Which of the following pseudocode is correct while calculating exponents?

- A) `int ans = 1; while(b>0){ if(b & 1) ans = ans * a; a = a * a; b <=<= 1; }`
- B) `int ans = 1; while(b>0){ if(b & 1) ans = ans * a; a = a * a; b <=<= 1; }`
- C) `int ans = 1; while(b>0){ if(b & 1) ans = ans * a; a = a * a; b >>= 1; }`
- D) `int ans = 1; while(b>=0){ if(b & 1) ans = ans * a; a = a * a; b >>= 1; }`

Answer: C)

Explanation: Correct implementation uses:

- `b & 1` to check if b is odd
 - `b >>= 1` to divide b by 2 (right shift)
 - Loop condition `b > 0` to avoid infinite loop
-

Question 13: Modulo Calculation

Evaluate the expression $6359 \bmod 320$.

- A) 681
- B) 811
- C) 3781
- D) 279

Answer: D) 279

Explanation: $6359 \div 320 = 19.871875$ $320 \times 19 = 6080$ $6359 - 6080 = 279$

Question 14: Modulo Arithmetic

$(12 \times 13) \% 5 = ?$

- A) 1
- B) 6
- C) 2
- D) 3

Answer: A) 1

Explanation: $12 \times 13 = 156$ $156 \bmod 5 = 1$ (since $156 = 31 \times 5 + 1$)

Question 15: GCD Properties

Which of the following is incorrect for calculating gcd of two elements a and b?

- A) $\gcd(a,b) = \gcd(a-b, b), a > b; \gcd(a,b) = \gcd(b-a, a), a < b;$
- B) $\gcd(a,b) = \gcd(b \% a, a)$
- C) $\gcd(a,b) = \gcd(a \% b, b)$
- D) None

Answer: D) None

Explanation: All given identities are correct for calculating GCD using the Euclidean algorithm. They are different forms of the same mathematical principle.

Question 16: Dereferencing Operator

The operator used for dereferencing is

- A) *
- B) &
- C) ->
- D) <

****Answer: A) *****

Explanation: The asterisk (*) is the dereferencing operator used to access the value that a pointer is pointing to.

Question 17: Pointer Declaration

Choose the right option: $\text{string}^* x, y;$

- A) x is a pointer to a string, y is a string
- B) y is a pointer to a string, x is a string
- C) both x and y are pointers to string types
- D) y is a pointer to a string

Answer: A) x is a pointer to a string, y is a string

Explanation: In C++, the * applies only to the variable it's directly attached to. So x is a pointer to string, and y is just a string variable.

Question 18: Illegal Pointer Declaration

Which of the following is illegal?

- A) `int *ip = 0;`
- B) `string s, *sp = 0;`
- C) `int i; double dp = &i;`
- D) None

Answer: C) `int i; double dp = &i;`

Explanation: This is illegal because you're trying to assign the address of an int (&i) to a variable dp of type double. There's a type mismatch - &i is of type int*, but dp is of type double.

Question 19: Pointer Assignment

What will happen in the following C++ code snippet?

cpp

```
int a = 100, b = 200;  
int *p = &a, *q = &b;  
p = q;
```

- A) b is assigned to a
- B) p now points to b
- C) a is assigned to b
- D) q now points to a

Answer: B) p now points to b

Explanation: The statement `p = q` assigns the value of q (which is the address of b) to p. So p now points to the same location as q, which is b.

Question 20: Array of Pointers Output

What will be the output of the following C++ code?

cpp


```
#include <iostream>
using namespace std;
int main() {
    int a = 5, b = 10, c = 15;
    int *arr[] = {&a, &b, &c};
    cout << arr[1];
    return 0;
}
```

- A) 5
- B) 10
- C) 15
- D) Address value of some element

Answer: D) Address value of some element

Explanation: `arr[1]` contains the address of variable b. Without the dereference operator (*), it prints the address, not the value 10.

Question 21: Function Pointer Syntax

The correct statement for a function that takes pointer to a float, a pointer to a pointer to a char and returns a pointer to a pointer to a integer is

- A) `int **fun(float**, char**)`
- B) `int *fun(float*, char*)`
- C) `int *fun(float*, char**)`
- D) `int **fun(float*, char**)`

Answer: D) `int **fun(float*, char**)`

Explanation:

- Takes pointer to float: `float*`
 - Takes pointer to pointer to char: `char**`
 - Returns pointer to pointer to int: `int**`
-

Question 22: Character Array Output

What will be the output of the following C++ code?

cpp

```
#include <iostream>
using namespace std;
int main() {
    char arr[20];
    int i;
    for(i = 0; i < 10; i++)
        *(arr + i) = 65 + i;
    *(arr + i) = '\0';
    cout << arr;
    return(0);
}
```

- A) ABCDEFGHIJ
- B) AAAAAAAAAA
- C) JJJJJJJJ
- D) AAAAAAJJJJ

Answer: A) ABCDEFGHIJ

Explanation: The loop fills arr[0] to arr[9] with ASCII values 65-74 (A-J), then adds null terminator at arr[10]. Output is "ABCDEFGHIJ".

Question 23: Pointer Arithmetic

What will be the output of the following C++ code?

cpp

```
#include <iostream>
using namespace std;
int main() {
    char *ptr;
    char Str[] = "abcdefg";
    ptr = Str;
    ptr += 5;
    cout << ptr;
    return 0;
}
```

- A) fg
- B) cdef

- C) defg
- D) abcd

Answer: A) fg

Explanation: `ptr += 5` moves the pointer to index 5, which points to 'f'. Printing from there gives "fg".

Question 24: Complex Pointer Function

What is the output of the complex pointer function?

cpp

```
int f(int x, int *py, int **ppz) {
    int y, z;
    **ppz += 1;    // c = c + 1 => c = 5
    z = **ppz;    // z = c => z = 5
    *py += 2;     // c = c + 2 => c = 7
    y = *py;      // y = c => y = 7
    x += 3;       // x = x + 3 => x = 4 + 3 = 7
    return x + y + z; // 7 + 7 + 5 = 19
}

int main() {
    int c, *b, **a;
    c = 4;
    b = &c;
    a = &b;
    cout << f(c, b, a);
    return 0;
}
```

- A) 19
- B) 20
- C) 21
- D) 18

Answer: A) 19

Explanation: Step by step: c starts as 4, becomes 5, then 7. Final calculation: x=7, y=7, z=5, total=19.

Question 25: Triple Pointer Output

What is the output?

cpp

```
#include<iostream>
using namespace std;
int main() {
    int ***r, **q, *p, i = 8;
    p = &i;
    (*p)++;    // i = 9
    q = &p;
    (**q)++;    // i = 10
    r = &q;
    cout << *p << " " << **q << " " << ***r;
    return 0;
}
```

- A) 9 9 9
- B) 8 8 8
- C) 10 10 10
- D) 9 10 11

Answer: C) 10 10 10

Explanation: All three expressions (*p, **q, ***r) ultimately point to the same variable i, which ends up with value 10.

Question 26: Function with Pointer Parameters

What is the output?

cpp

```

#include <iostream>
using namespace std;
void Q(int z) {
    z += z;      // z = z + z
    cout << z << " ";
}
void P(int *y) {
    int x = *y + 2; // x = 5 + 2 = 7
    Q(x);          // Calls Q(7) -> outputs 14
    *y = x - 1;    // *y = 6
    cout << x << " "; // outputs 7
}
int main() {
    int x = 5;
    P(&x);
    cout << x;      // outputs 6
    return 0;
}

```

- A) 7 6 14
- B) 14 7 6
- C) 14 7 5
- D) 14 6 5

Answer: B) 14 7 6

Explanation: Q(7) outputs 14, then P outputs 7, finally main outputs modified x=6.

Question 27: Array Access Methods

What is the output?

```

cpp

#include<iostream>
using namespace std;
int main() {
    char st[] = "ABCD";
    for(int i = 0; st[i] != '\0'; i++) {
        cout << st[i] << *(st + i) << *(i + st) << i[st];
    }
    return 0;
}

```

- A) Compilation Error
- B) Empty
- C) ABCD
- D) AAAABBBBCCCCDDDD

Answer: D) AAAABBBBCCCCDDDD

Explanation: All four expressions (`st[i]`, `*(st+i)`, `*(i+st)`, `i[st]`) are equivalent ways to access `st[i]`. Each iteration prints the same character 4 times.

Question 28: Const Pointer Error

What is the output?

```
cpp
#include <iostream>
using namespace std;
int main(){
    int p = 5;
    int const *q = &p; // Fixed syntax
    p++;
    cout << p << endl;
    return 0;
}
```

- A) Error
- B) 6
- C) 5
- D) None

Answer: A) Error (due to original syntax error)

Explanation: The original code had invalid syntax `int const 'q = &p;`. The apostrophe makes it a syntax error.

Question 29: Modifying Const Variable

What is the output?

```
cpp
```

```
#include <iostream>
using namespace std;
int main() {
    int const p = 5;
    cout << ++p;
    return 0;
}
```

- A) 5
- B) 6
- C) Garbage
- D) Error

Answer: D) Error

Explanation: `p` is declared as `const int`, so `++p` attempts to modify a constant, which results in a compile-time error.

Question 30: Dynamic 2D Array Creation

How to Dynamically create a Two Dimensional Array in C++?

- A) `int **arr = new int[10][10];`
- B) `int **arr = new int[10];`
- C) `int **arr = new int[][10];`
- D) None

Answer: D) None

Explanation: The correct way requires two steps:

```
cpp
int **arr = new int*[10];    // Create array of pointers
for(int i = 0; i < 10; i++) {
    arr[i] = new int[10];    // Create each row
}
```

None of the given options correctly creates a full 2D array dynamically.

Summary

This collection covers essential C++ concepts including:

- Prime number algorithms and complexity analysis
- Pointer operations and dereferencing
- Dynamic memory allocation
- Function parameters and return types
- Array operations and pointer arithmetic
- Const correctness
- Modular arithmetic and bit operations

Each question tests practical understanding of C++ syntax and memory management concepts commonly encountered in programming interviews and coursework.